HotChocolate (GraphQL)

Architektur, Patterns und Democode

Detaillierte Ausarbeitung • .NET 8 LTS • Code-First

Stand: April 2026

Inhaltsverzeichnis

Inhaltsverzeichnis 2
1. Einleitung & Zielsetzung 3
2. Technologiestack 3
3. Grundprinzipien 3
    3.1 Resolver sind duenn 3
    3.2 Business-Logik gehoert in Services 3
    3.3 Alles async, CancellationToken durchreichen 4
    3.4 IQueryable statt IEnumerable 4
    3.5 Input-Objekte statt Primitive-Flood 4
4. Projektstruktur 4
    Ordnerstruktur im Democode 4
5. Domain Model 5
6. Application Service 5
7. GraphQL Types (ObjectType) 6
8. Queries - Read-Zugriff 6
9. Mutations - Write-Zugriff 7
10. Subscriptions - Realtime 7
11. DataLoader - N+1 vermeiden 7
12. Authentication & Authorization 8
13. Error Handling 8
14. Program.cs - Composition Root 9
15. Sicherheit & Performance 9
    15.1 Query Limits 9
    15.2 Persisted Queries 10
    15.3 Introspection 10
16. Testing-Strategie 10
17. Anti-Patterns 10
18. Deployment-Checkliste 11
19. Mentales Modell 11
20. Democode-Wegweiser 11

1. Einleitung & Zielsetzung

HotChocolate ist der mit Abstand am weitesten verbreitete GraphQL-Server in der .NET-Welt. Die Bibliothek setzt konsequent auf ein Code-First-Modell, integriert sich nativ in das ASP.NET Core-Hostingmodell und liefert Middleware fuer Filtering, Sorting, Paging, Projections, Authorization sowie einen batch-optimierten DataLoader gleich mit.

Diese Ausarbeitung beschreibt eine produktionsnahe Architektur fuer GraphQL-APIs mit HotChocolate v14 und .NET 8 LTS. Sie gibt klare Richtlinien, wie Resolver, Services und Domaene geschichtet werden, welche Fehler und Anti-Patterns vermieden werden muessen, und liefert ein vollstaendiges, lauffaehiges Democode-Projekt.

> *Der gesamte Democode liegt unter HotChocolate.Demo/ im gleichen Ordner wie diese Datei und laesst sich mit "dotnet run" starten.*

2. Technologiestack

Komponente	Entscheidung	Begruendung
Runtime	.NET 8 LTS	Aktuelle LTS mit Support bis 11/2026
GraphQL-Server	HotChocolate v14	Federation v2, .NET 8 optimiert
Hosting	ASP.NET Minimal Hosting	Weniger Boilerplate, klare Startup-Sequenz
Schema-Ansatz	Code-First	Refactoring-sicher, IDE-Support, Typsicherheit
Persistence	EF Core 8 (InMemory/SQL)	Produktive Integration mit Projections
Tooling	Banana Cake Pop	Mitgeliefertes IDE unter /graphql
Subscriptions	InMemory / Redis	InMemory fuer Dev, Redis fuer Production

3. Grundprinzipien

Die folgenden Prinzipien ziehen sich durch den kompletten Democode und sollten in jedem produktiven HotChocolate-Projekt eingehalten werden.

3.1 Resolver sind duenn

- Ein Resolver orchestriert, er implementiert keine Fachlichkeit.
- Er reicht Parameter an einen Application Service weiter und kapselt das Ergebnis gegebenenfalls in ein Payload.
- Das macht Resolver trivial zu lesen und ermoeglicht den identischen Service unter REST, gRPC oder einem Worker zu verwenden.

3.2 Business-Logik gehoert in Services

- Application Services sind framework-unabhaengig - kein HotChocolate, kein ASP.NET, keine Attribute.
- Sie sind die natuerliche Stelle fuer Validierung, Transaktionen und Side-Effects.
- Tests koennen den Service in-process instanziieren, ohne einen HTTP-Kontext aufzubauen.

3.3 Alles async, CancellationToken durchreichen

- Niemals Task.Run als Schein-Asynchronitaet - das belegt Threadpool-Slots ohne Nutzen.
- Cancellation Tokens werden von HotChocolate automatisch bereitgestellt und sollten bis in den DbContext durchgereicht werden.

3.4 IQueryable statt IEnumerable

- Queries geben IQueryable<T> zurueck, damit HotChocolate Projections, Filter, Sorting und Paging in SQL uebersetzt.
- Mit IEnumerable landet alles im Arbeitsspeicher - bei 10.000+ Datensaetzen tierisch teuer.

3.5 Input-Objekte statt Primitive-Flood

- Mutations akzeptieren genau ein Input-Objekt (AddBookInput, UpdateBookInput).
- Neue Felder koennen additiv erweitert werden, ohne das Schema zu brechen.
- Records sind ideal: immutable, ohne Boilerplate, mit struktureller Gleichheit.

4. Projektstruktur

Empfohlene Aufteilung in drei Schichten. Die Abhaengigkeit zeigt ausschliesslich nach innen.

Schicht	Verantwortung	Kennt
Domain	Entitaeten, Wertobjekte, Domain Events, Business-Regeln	nichts
Application	Use-Cases, Orchestrierung, Validierung, Transaktionen	Domain
GraphQL	Schema, Resolver, Types, DataLoader, Auth-Policies	Domain + Application

Wichtig: Domain und Application kennen HotChocolate nicht. Es gibt keine [GraphQLNonNull]-Attribute auf Entitaeten - dafuer sind die ObjectTypes zustaendig.

Ordnerstruktur im Democode

```
HotChocolate.Demo/
├─ Domain/
│   ├─ Book.cs
│   ├─ Author.cs
│   └─ DomainExceptions.cs
├─ Data/
│   ├─ LibraryDbContext.cs
│   └─ DatabaseSeeder.cs
├─ Application/
│   ├─ BookService.cs
│   └─ AuthorService.cs
├─ GraphQL/
│   ├─ Query.cs
│   ├─ Mutation.cs
│   ├─ Subscription.cs
│   ├─ DomainErrorFilter.cs
```

```
│  ├─ Types/BookType.cs
│  ├─ Types/AuthorType.cs
│  ├─ Inputs/BookInputs.cs
│  ├─ Payloads/BookPayloads.cs
│  └─ DataLoaders/AuthorByIdDataLoader.cs
└─ Program.cs
```

5. Domain Model

Die Domaene ist der Ort der Wahrheit. Hier liegt die Sprache der Fachlichkeit - nichts anderes. Typische Eigenschaften:

- immutable: init-only Setter, keine Setter-Magie.
- keine Attribute: kein [Required], kein [GraphQLNonNull]. Das verhindert Framework-Lock-in.
- keine Navigation-Properties, wenn die Beziehung im GraphQL-Layer per DataLoader aufgeloest wird.

Beispiel Book aus dem Democode:

```
public sealed class Book
{
    public Guid Id { get; init; }
    public string Title { get; init; } = default!;
    public string Isbn  { get; init; } = default!;
    public DateOnly PublishedOn { get; init; }
    public int PageCount { get; init; }
    public Genre Genre { get; init; }
    public Guid AuthorId { get; init; }   // Fremdschluessel, keine Navigation
}
```

6. Application Service

Der Application Service kapselt den kompletten Use-Case. Er ist die einzige Stelle, an der Validierung, Autorisierung auf Datenebene und SaveChanges stattfinden.

Drei Methoden-Typen wiederholen sich in jedem Service:

1. Read-Queries geben IQueryable<T> zurueck - HotChocolate uebersetzt die GraphQL-Selektion direkt in SQL.
2. Single-Record-Reads sind asynchron und nutzen CancellationToken.
3. Writes nehmen Parameter bzw. ein internes Input-DTO, validieren, persistieren und geben die Entitaet zurueck.

Auszug aus BookService:

```
public IQueryable<Book> QueryBooks()
    => _dbFactory.CreateDbContext().Books.AsNoTracking();


public async Task<Book> AddAsync(...)
{
    await using var db = await _dbFactory.CreateDbContextAsync(ct);
```

```
    if (string.IsNullOrWhiteSpace(title)) throw new ValidationException(...);
    if (!await db.Authors.AnyAsync(a => a.Id == authorId, ct))
        throw new AuthorNotFoundException(authorId);
    // ...
}
```

7. GraphQL Types (ObjectType)

HotChocolate kann Typen implizit ableiten. Fuer produktive Schnittstellen lohnt es sich jedoch fast immer, die Typen explizit zu schreiben. Gruende:

- Volle Kontrolle ueber Nullability (NonNullType<StringType>).
- Stabile Namen - Umbenennungen in der Domaene brechen nicht das Schema.
- Feinere Dokumentation per Description - landet in Banana Cake Pop und in generierten Clients.
- Eigene Resolver fuer virtuelle Felder (author, books, createdBy).
- Vorbereitung fuer Apollo Federation v2.

Beispiel BookType:

```
public sealed class BookType : ObjectType<Book>
{
    protected override void Configure(IObjectTypeDescriptor<Book> d)
    {
        d.BindFieldsImplicitly();
        d.Field(b => b.Id).Type<NonNullType<IdType>>();
        d.Field(b => b.Title).Type<NonNullType<StringType>>();

        d.Ignore(b => b.AuthorId);          // intern ausblenden
        d.Field("author")                   // virtuelles Feld
           .Type<NonNullType<AuthorType>>()
           .ResolveWith<BookResolvers>(r =>
               r.GetAuthorAsync(default!, default!, default));
    }
}
```

8. Queries - Read-Zugriff

Die Middleware-Reihenfolge ist kritisch. HotChocolate prozessiert sie inside-out, deshalb muessen die Attribute in genau dieser Reihenfolge stehen:

```
[UsePaging(IncludeTotalCount = true, DefaultPageSize = 25)]
[UseProjection]
[UseFiltering]
[UseSorting]
public IQueryable<Book> GetBooks([Service] BookService s)
    => s.QueryBooks();
```

Attribut	Wirkung

UsePaging	Relay-konformes Cursor-/Offset-Paging; IncludeTotalCount nur aktivieren, wenn Client es braucht
UseProjection	Baut aus der GraphQL-Selektion eine SELECT-Projektion - kein SELECT *
UseFiltering	Ermoeglicht where-Klauseln (eq, contains, gt, in, and, or)
UseSorting	Ermoeglicht order: [{ field: ASC|DESC }]

> *Regel: UsePaging muss zuerst stehen, sonst erhaelt der Client alle Datensaetze vor der Paginierung.*

9. Mutations - Write-Zugriff

Mutations folgen einer klaren Konvention:

- Exakt ein Input-Objekt als Parameter (Relay-Style).
- Rueckgabe ist ein Payload-Record, der sich erweitern laesst, ohne das Schema zu brechen.
- Fehler werden als typisierte Exceptions aus dem Service geworfen und vom DomainErrorFilter in GraphQL-Errors uebersetzt.

```
[Authorize(Policy = "librarian")]
public async Task<AddBookPayload> AddBookAsync(
    AddBookInput input,
    [Service] BookService service,
    [Service] ITopicEventSender events,
    CancellationToken ct)
{
    var book = await service.AddAsync(...);
    await events.SendAsync(nameof(Subscription.OnBookAdded), book, ct);
    return new AddBookPayload(book);
}
```

10. Subscriptions - Realtime

Subscriptions werden ueber WebSockets oder SSE ausgeliefert. HotChocolate stellt dafuer ITopicEventSender (in Mutations) und ITopicEventReceiver (in Subscriptions) bereit.

```
public sealed class Subscription
{
    [Subscribe]
    [Topic]
    public Book OnBookAdded([EventMessage] Book book) => book;
}
```

Fuer die Produktion empfiehlt sich ein persistenter Transport wie Redis:

```
.AddRedisSubscriptions(_ => ConnectionMultiplexer.Connect("localhost"))
```

11. DataLoader - N+1 vermeiden

Bei GraphQL ist das N+1-Problem strukturell: 100 Buecher im Query bedeuten ohne Batching 100 einzelne Autor-Abfragen. DataLoader sammelt in einem Request alle Ids und loest sie in einer einzigen Batch-Abfrage auf.

```
public sealed class AuthorByIdDataLoader(
    AuthorService svc, IBatchScheduler batch, DataLoaderOptions opt)
    : BatchDataLoader<Guid, Author>(batch, opt)
{
    protected override Task<IReadOnlyDictionary<Guid, Author>>
        LoadBatchAsync(IReadOnlyList<Guid> keys, CancellationToken ct)
        => svc.GetByIdsAsync(keys, ct);
}
```

Loader-Typ	Signatur	Einsatz
BatchDataLoader<K,V>	Key → Value	1:1-Beziehung (Buch hat genau einen Autor)
GroupedDataLoader<K,V>	Key → IEnumerable<V>	1:N-Beziehung (Autor hat viele Buecher)
CacheDataLoader<K,V>	Key → Value (einzeln)	Caching ohne Batching
StatefulBatchDataLoader	Key → Value + State	Cross-Request-Cache via Redis

12. Authentication & Authorization

HotChocolate integriert sich in ASP.NET Core Authentication. Damit das [Authorize]-Attribut auf Resolvern greift, muss .AddAuthorization() im GraphQL-Builder registriert sein.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => { /* Key, Issuer, Audience */ });

builder.Services.AddAuthorization(o =>
{
    o.AddPolicy("reader",    p => p.RequireAuthenticatedUser());
    o.AddPolicy("librarian", p => p.RequireRole("librarian","admin"));
});

builder.Services.AddGraphQLServer()
    .AddAuthorization()
    .AddQueryType<Query>();
```

Attribute koennen auf Query-/Mutation-Methoden gelegt werden oder auf einzelne Felder eines ObjectType.

Empfehlung: Authorization immer auf die kleinste sinnvolle Einheit - bei sensitiven Feldern lieber pro Feld als pro Type.

13. Error Handling

In HotChocolate landet jede nicht gefangene Exception als GraphQL-Error im Response. In Production duerfen aber keine Stack Traces nach aussen - und Clients wollen strukturierte Fehler mit einem stabilen Code.

Die Loesung ist ein IErrorFilter, der Domaenen-Exceptions in Errors mit Code und Extensions uebersetzt:

```
public sealed class DomainErrorFilter : IErrorFilter
{
    public IError OnError(IError error) => error.Exception switch
    {
        BookNotFoundException ex => error
            .WithMessage(ex.Message)
            .WithCode("BOOK_NOT_FOUND")
            .SetExtension("bookId", ex.BookId),

        ValidationException ex => error
            .WithMessage(ex.Message)
            .WithCode("VALIDATION_FAILED"),

        _ => error
    };
}
```

14. Program.cs - Composition Root

Die gesamte Konfiguration soll explizit sein. Keine Magie, keine versteckten Side-Effects. Jeder Aufruf hat einen nachvollziehbaren Effekt:

```
builder.Services.AddGraphQLServer()
    .AddQueryType<Query>()
    .AddMutationType<Mutation>()
    .AddSubscriptionType<Subscription>()
    .AddType<BookType>().AddType<AuthorType>()
    .AddErrorFilter<DomainErrorFilter>()
    .AddFiltering().AddSorting().AddProjections()
    .AddAuthorization()
    .AddInMemorySubscriptions()
    .AddMaxExecutionDepthRule(10)
    .AddDataLoader<AuthorByIdDataLoader>();

app.MapGraphQL("/graphql");
```

15. Sicherheit & Performance

15.1 Query Limits

- MaxExecutionDepthRule: verhindert rekursiv tiefe Queries (z.B. book → author → books → author → …)
- Cost Analysis: HotChocolate v14 unterstuetzt das Computing von Query-Kosten und das Ablehnen teurer Abfragen.

- Timeouts pro Request: ModifyRequestOptions(opt => opt.ExecutionTimeout = TimeSpan.FromSeconds(30)).

15.2 Persisted Queries

In Production sollten Clients nur vorregistrierte Queries nutzen koennen. Damit ist die Angriffsflaeche auf praktisch Null reduziert.

```
.AddReadOnlyFileSystemQueryStorage("./persisted_queries")
.UsePersistedQueryPipeline()
```

15.3 Introspection

Oeffentlich exponierte APIs ohne Schema-Veroeffentlichungspflicht sollten Introspection deaktivieren:

```
.AllowIntrospection(builder.Environment.IsDevelopment())
```

16. Testing-Strategie

Test-Art	Was wird getestet	Tooling
Unit	Application Service isoliert	xUnit + FakeItEasy/Moq
Schema-Snapshot	Schema-Stabilitaet	HotChocolate.Tests + VerifyXunit
Integration	Server-in-Memory + Query	WebApplicationFactory
Contract	Client-Generator laeuft durch	StrawberryShake build

Beispiel eines Schema-Snapshot-Tests (verifiziert, dass ein Pull Request das Schema nicht unbeabsichtigt aendert):

```
[Fact]
public async Task Schema_ist_stabil()
{
    var schema = await new ServiceCollection()
        .AddGraphQLServer()
        .AddQueryType<Query>()
        .BuildSchemaAsync();

    schema.Print().MatchSnapshot();
}
```

17. Anti-Patterns

Was	Problem	Fix
Business-Logik im Resolver	Nicht testbar, nicht wiederverwendbar	Service extrahieren
IEnumerable aus Queries	Lädt alles in RAM	IQueryable zurueckgeben
DbContext direkt im Resolver	Concurrency-Probleme, schwer testbar	DbContextFactory + Service

Was	Problem	Fix
Viele Primitive in Mutations	Breaking Changes bei neuen Feldern	Input-Objekt
Lazy Loading auf Navigationen	Zuverlaessig N+1	DataLoader
Include() ueber alles	JOIN-Explosion	UseProjection + DataLoader
Keine Query-Limits	DoS durch tiefe Queries	MaxExecutionDepthRule + Cost
Introspection in Prod	Schema leakt an Angreifer	AllowIntrospection(false)
Hardcoded JWT-Key	Credential-Leak bei Git Push	IConfiguration + Secret Manager

18. Deployment-Checkliste

4. appsettings.Production.json enthaelt keine Secrets - alle Secrets ueber Environment Variables oder Key Vault.
5. IncludeExceptionDetails = false im Produktions-Environment.
6. AllowIntrospection = false bei nicht-oeffentlichen APIs.
7. Persisted Queries aktiviert und im Build-Prozess befuellt.
8. MaxExecutionDepthRule und Cost Rules konfiguriert.
9. Subscriptions auf Redis/Postgres umgestellt (InMemory nur fuer Dev).
10. OpenTelemetry-Tracing konfiguriert (AddInstrumentation fuer HotChocolate).
11. Healthchecks /health und Readiness /ready registriert.
12. Rate-Limiting vor dem GraphQL-Endpunkt (ASP.NET Rate Limiter).
13. CI-Pipeline fuehrt Schema-Snapshot-Test aus.

19. Mentales Modell

> *GraphQL ist kein Datenbank-Abfrage-DSL, sondern ein API-Vertrag mit starker Typisierung. HotChocolate ist die Middleware-Pipeline, die diesen Vertrag ausfuehrt.*

Begriff	Analogie
Resolver	Controller (MVC)
Service	Use-Case / Application Service
Domain	Fachliche Wahrheit
Schema	API-Vertrag mit dem Frontend
DataLoader	Request-lokaler Batch + Cache
Middleware	Filter/Sort/Paging-Pipeline vor dem Resolver

20. Democode-Wegweiser

Das beiliegende Democode-Projekt (HotChocolate.Demo/) laesst sich direkt starten:

```
cd HotChocolate.Demo
dotnet restore
dotnet run

# Banana Cake Pop oeffnet sich unter http://localhost:5071/graphql
```

Die Datei Examples.graphql enthaelt vorbereitete Queries, Mutations und Subscriptions. In Banana Cake Pop kannst du sie per Copy/Paste direkt ausprobieren.

Der Code bildet 1:1 die Patterns dieses Dokuments ab:

- Layered Architecture mit Domain / Application / GraphQL.
- Code-First ObjectTypes mit expliziter Nullability und Descriptions.
- Queries mit Paging, Filter, Sorting und Projections.
- Mutations mit Input/Payload-Records und Authorization-Policies.
- Subscriptions ueber ITopicEventSender.
- DataLoader (Batch und Grouped) zur N+1-Vermeidung.
- DomainErrorFilter fuer strukturierte GraphQL-Errors.
- EF Core via IDbContextFactory<T> fuer parallele Resolver.